PROJECT SYNOPSIS

ON

AI EXAM PROCTORING SYSTEM

A Multi-Tenant SaaS Platform for AI-Driven Remote Examination Integrity

In partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

(Computer Science & Engineering)

Submitted by

Name	Roll No	Section	Role
Anurag Kumar	11233016	7A1	Backend Developer
Anchal	11232539	7A1	Frontend Developer
Aman	11232530	7A2	AI / ML
Supriya	11233040	7A1	AI / ML

Under the supervision of

Prof. Dr. Abhishek Bhattacharya

CSE Department

M.M. ENGINEERING COLLEGE

MAHARISHI MARKANDESHWAR (DEEMED TO BE UNIVERSITY)

(NAAC accredited Grade 'A++' University)

MULLANA, AMBALA, HARYANA

July 2026 – Nov 2026

1. Abstract

Online education has grown fast over the last few years, but exam integrity hasn't really kept pace with it. Most online exam platforms today can't tell if the person on screen is actually the registered candidate, whether someone else is sitting just out of frame, or whether the student has switched tabs to look up an answer. Human invigilators can't watch hundreds of students at once either, so a lot of institutions are stuck choosing between exams they can't fully trust and proctoring that doesn't scale.

This project works on that problem directly. It proposes an AI-based exam proctoring platform, built as a multi-tenant SaaS product, so one deployment can serve many colleges, universities, or companies at the same time. Each institution runs as its own isolated Organization, with its own admins, exam creators, proctors, and students, while the platform owner manages the whole system from above as Super Admin. During an exam, the system watches the webcam and microphone continuously, checking for things like more than one face in frame, no face at all, the candidate looking away too often, objects that shouldn't be there, or more than one voice being picked up. Browser behavior is tracked too — tab switching and exiting fullscreen are both logged. None of these checks work as a silent black box; every flagged event is stored with a timestamp, a confidence score, and the evidence behind it, so a human reviewer can actually see why something was flagged.

On the technical side, the system is split into three layers: a React/Next.js frontend, a Spring Boot backend broken into separate microservices by domain, and a Python FastAPI service that handles the AI work itself — MediaPipe for face and gaze tracking, ArcFace for recognition, YOLOv8 for detecting objects, and pyannote.audio for figuring out how many people are speaking. PostgreSQL handles storage, Redis handles caching, and WebSockets keeps everything updating live during the exam. Once the exam ends, all of this feeds into a single Trust Score, backed by a full evidence report, so proctors and administrators aren't just trusting a number — they can see exactly what happened and make the final call themselves.

2. Introduction

Online exams have made testing more flexible and accessible, there's no denying that. But that same flexibility opens the door to problems that a physical exam hall simply doesn't have to deal with. Most institutions still rely on manual invigilation for their online exams, and honestly, that approach was never built for this scale. It's expensive to staff, hard to grow, and no proctor — no matter how attentive — can keep an eye on every single candidate at every single moment. So things slip through: someone else sitting in for the real candidate, notes or a second device just out of frame, a person quietly feeding answers in the background, a browser tab switched open to search something up. These aren't rare edge cases either — they're risks that institutions and certification bodies deal with constantly, and mostly without a real way to catch them.

This project, the AI Exam Proctoring System, was built to close that gap. It's a web-based platform that uses AI to watch over exams in real time, through the candidate's webcam and microphone, checking that the person taking the exam is actually who they claim to be and flagging behavior that looks suspicious. But it doesn't try to play judge on its own. Instead of kicking a student out or auto-failing them the moment something looks off, the system just records what happened — with a timestamp and evidence attached — and calculates a trust score. The actual decision on whether that flag matters gets left to a human, a proctor or an administrator, which feels like the right way to handle something as high-stakes as academic integrity.

What makes this different from a typical single-institution proctoring tool is really the architecture behind it. It's built as a multi-tenant SaaS platform, meaning a Super Admin can onboard entirely separate organizations onto the same system, and each one gets its own Organization Admin, Exam Creators, Proctors, and Students, all working within their own fully isolated slice of data (more on this in Section 6). In practice, that means a college, a university, or a company doesn't need to build and maintain its own AI detection pipeline from scratch just to get proctoring that actually works — they can plug into one shared platform instead. How that's structured underneath, both the system architecture and how the data is organized, is covered in Sections 7 and 8.

---

3. Problem Statement

Talk to almost any institution running online exams today, and you'll hear versions of the same complaints:

● **Human invigilation just doesn't scale.** Watching every candidate continuously requires proctors, and more candidates simply means more proctors, more cost, more scheduling headaches. There's no getting around that math.

● **The tools institutions already use are too easy to get past.** A snapshot every few minutes, or a face check only at login — that leaves most of the exam completely unwatched, and anyone who's thought about it for five minutes can find a way around it.

● **Building this properly in-house is a huge lift.** Face detection, gaze tracking, object recognition, audio monitoring — getting all of that working reliably isn't a side project. Most institutions don't have the time, money, or in-house expertise to build and maintain something like that themselves, and honestly, most shouldn't have to.

● **The commercial options that do exist aren't built for smaller players.** A lot of them are either locked to a single institution's use or priced like enterprise software, which puts self-serve

access out of reach for smaller colleges or certification bodies that just want to run their own exams without a six-figure contract.

● **When something does get flagged, there's often not much to back it up.** No timestamp, no confidence score, no clip — just a proctor's memory or maybe one screenshot. That's a weak foundation when a student disputes a decision, and it puts institutions in an awkward spot.

Put all of that together, and it points pretty clearly toward one kind of solution: a multi-tenant SaaS platform. Build the AI-driven proctoring pipeline once, run it well, and let any number of institutions subscribe and operate independently on top of it — sharing the same underlying inference engine, infrastructure, and cost base instead of each one reinventing it from scratch.

4. Objectives

At its core, this project is about building a platform that makes remote exams fair and trustworthy — for the student taking it and the institution grading it — while letting many different organizations run on the same shared system without stepping on each other's data. To get there, the work breaks down into a few concrete goals:

● **Getting the multi-tenant piece right.** A Super Admin should be able to bring new organizations onto the platform, turn their access on or off, and manage what subscription plan they're on — all while each organization's data stays completely separate from every other one, with its own admin structure sitting on top.

● **Locking down who can access what.** Login needs to be secure and role-aware, using JWT tokens that know not just who the user is but which organization they belong to and what role they hold — Super Admin, Org Admin, Exam Creator, Proctor, or Student — so nobody can accidentally (or deliberately) reach data outside their scope.

● **Verifying identity before the exam even starts.** Using ArcFace, the system checks the candidate's live webcam feed against a reference photo they registered earlier, so someone else can't sit the exam in their place.

● **Keeping watch throughout the entire exam, not just at the start.** Both the webcam and microphone stay active and get analyzed continuously, looking out for more than one face showing up, the candidate's face disappearing altogether, their gaze wandering away from the screen too much, objects that shouldn't be there, conversations happening in the background, tabs being switched, or the browser leaving fullscreen.

● **Making every flag mean something.** Each violation gets logged with a timestamp, a confidence score, and actual supporting evidence — not just a vague alert. All of that feeds into a trust score, and by the end of the exam, there's a full report a proctor or admin can actually review and act on.

● **Giving exam creators real tools to work with.** They should be able to build out question banks, decide how long an exam runs and which proctoring checks apply, and schedule or assign exams to the right students.

● **Building something that could actually run as a product, not just a demo.** The whole thing is put together as a modular microservice system — Spring Boot, FastAPI, and React/Next.js working together — with subscription billing baked in from the start, so it's positioned to work as a real commercial SaaS platform, not something that only makes sense as a college project.

5. Literature Review

Looking at what's already out there — both in industry and academic research — this project's design pulls from three fairly distinct areas.

❖ **Commercial proctoring platforms.** Tools like ProctorU, Examity, and Honorlock have been around for a while, and they mostly work by combining live human proctors with some automated flagging on top — a face match when the candidate logs in, maybe basic motion detection during the exam. What's worth borrowing from them is the underlying philosophy: flag suspicious behavior, don't auto-fail the student. That's a principle this project leans on too. But these platforms are largely built and priced for big enterprise clients, and self-serve onboarding for smaller institutions is pretty much an afterthought, if it exists at all. That's a real gap, and it's one this project tries to address head-on.

❖ **Academic research on the individual detection problems.** If you look at the pieces separately, the research is actually pretty mature. Face verification using deep embeddings is well established — ArcFace, with its additive angular margin loss (Deng et al., 2019), is more or less the standard approach now. Facial landmark detection and head-pose estimation have solid groundwork too, from Bulat & Tzimiropoulos (2017) through to MediaPipe's real-time face mesh, which is what a lot of production systems lean on today. Object detection has its own well-worn path — Redmon & Farhadi's YOLO9000 is really where the single-shot detection approach took off, and it's the lineage YOLOv8 (used in this project) builds on. And for figuring out how many people are speaking, pyannote.audio (Bredin et al., 2020) covers speaker diarization reliably. What's missing, though, is a prototype that actually pulls all of these pieces together into one real-time pipeline, tied into a single trust-score model — and almost none of them touch multi-tenancy at all. That combination is really the gap this project is trying to fill.

❖ **Multi-tenant SaaS design patterns.** None of the multi-tenancy approach here is new on its own — a shared database with a tenant-discriminator column (in this case, an

organization_id foreign key), role-based access control layered on top of that tenancy, and breaking the backend into services behind an API gateway — these are all well-established SaaS patterns, used across plenty of industries already. What this project does is apply that same pattern to exam proctoring specifically, which isn't something that shows up much in the existing work.

So really, the contribution here isn't a new AI technique or a novel model — it's the combination: multi-modal AI detection, wrapped in a proper multi-tenant SaaS architecture, with billing built in from the start. That combination, applied specifically to exam proctoring, is what's largely absent from both the commercial and academic landscape right now.

6. Multi-Tenant Role Hierarchy

The platform is a multi-tenant SaaS product: the project team's own organization operates as the Super Admin, and every college, university, or company that subscribes becomes an independent Organization with its own admins, exam creators, proctors, and students, fully isolated from other tenants' data.

Role	Scope	Key Permissions
Super Admin	Platform-wide (own company)	Create organizations, manage subscription plans, enable/disable organizations, platform-wide analytics, global settings, billing, AI model version management
Organization Admin	One organization (e.g., a college)	Create Exam Creators & Proctors, invite Students, create departments, view organization reports, manage organization settings
Exam Creator	Within an organization	Create question banks, create exams, configure duration, set AI proctoring rules, publish exams
Proctor (Invigilator)	Assigned exam sessions	Monitor live exams, receive AI alerts, pause/terminate exam, approve suspicious cases, chat with students
Student	Own exam attempts	Login, identity verification, attempt exam, submit exam, view result

7. System Architecture

The platform follows a three-tier, service-oriented architecture. The presentation tier is a React.js/Next.js single-page application; the application tier is a set of Spring Boot (Java) microservices, one per bounded context (auth, organization, user, role, question-bank, exam, exam-session, proctoring, violation, report, notification, dashboard, audit, storage); and the AI inference tier is an independent Python FastAPI microservice, called only through the backend's proctoring module. PostgreSQL is the system of record, Redis handles caching and real-time pub/sub for WebSocket alerts, and evidence media is written to S3-compatible object storage.

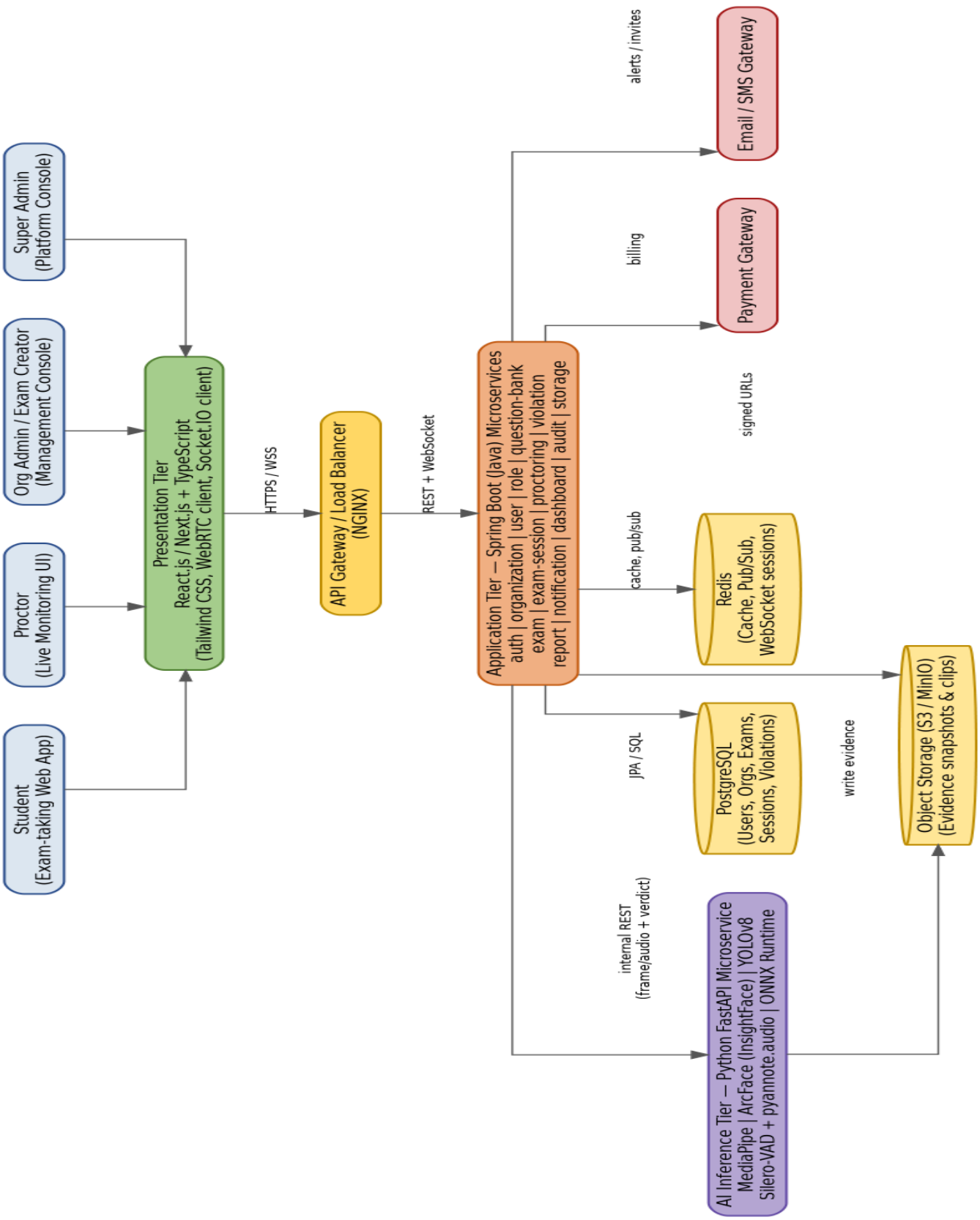


Figure 1 — Three-Tier Multi-Tenant System Architecture

Two third-party integrations sit at the platform boundary and are owned entirely by the backend: a Payment Gateway for subscription billing (Super Admin flow) and an Email/SMS Gateway for exam invitations and proctor alerts. Neither the frontend nor the AI/ML service integrates with these directly.

8. Database Design (ER Diagram)

The core entity-relationship model — Users, Exams, Questions, Exam Assignments, Exam Sessions, Answers, and Violations — sits underneath a tenancy layer that scopes nearly every table to an organization_id, plus supporting tables for subscriptions, notifications, and auditing (not shown below for clarity). Every violation links to its evidence media, and every exam session accumulates a trust_score computed from its associated violations at submission.

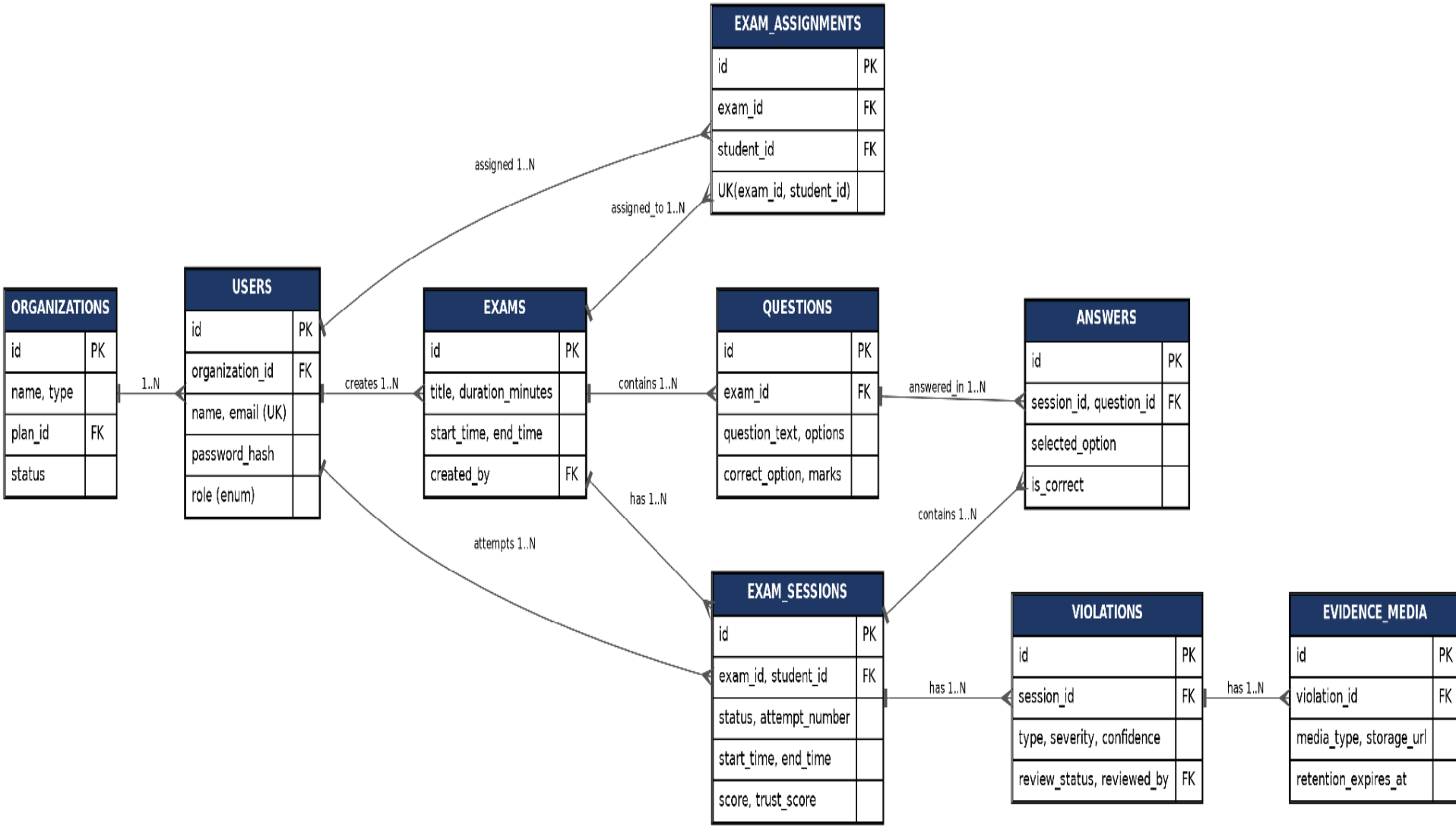


Figure 2 — Entity-Relationship Diagram (Core Schema)

9. Data Flow Diagrams (DFD)

The following diagrams model how data moves through the system at increasing levels of detail: a Level 0 context view, a Level 1 view of the eight major subsystems, and Level 2 decomposition of the two processes central to the exam-taking and AI-proctoring experience.

9.1 DFD Notation Key

Symbol	Meaning	Example
Rectangle	External Entity — a person or outside system	Student, Proctor, Payment Gateway
Ellipse (N.0)	Process — a function that transforms or routes data	5.0 AI Inference & Violation Detection
Cylinder (DN)	Data Store — a persistent table the process reads/writes	D9 Exam Sessions
Labeled arrow	Data Flow — the specific data moving between elements	"live frame/audio stream"

9.2 Level 0 — Context Diagram

The entire platform is treated as a single process, showing only who and what sits outside the system boundary.

Figure 3 — Level 0 Context Diagram

9.3 Level 1 —

Major Subsystems

The single Level 0 process is broken into the eight subsystems that map directly onto the Spring Boot modules and the AI/ML service — the shared source of truth for who-calls-what across the team.

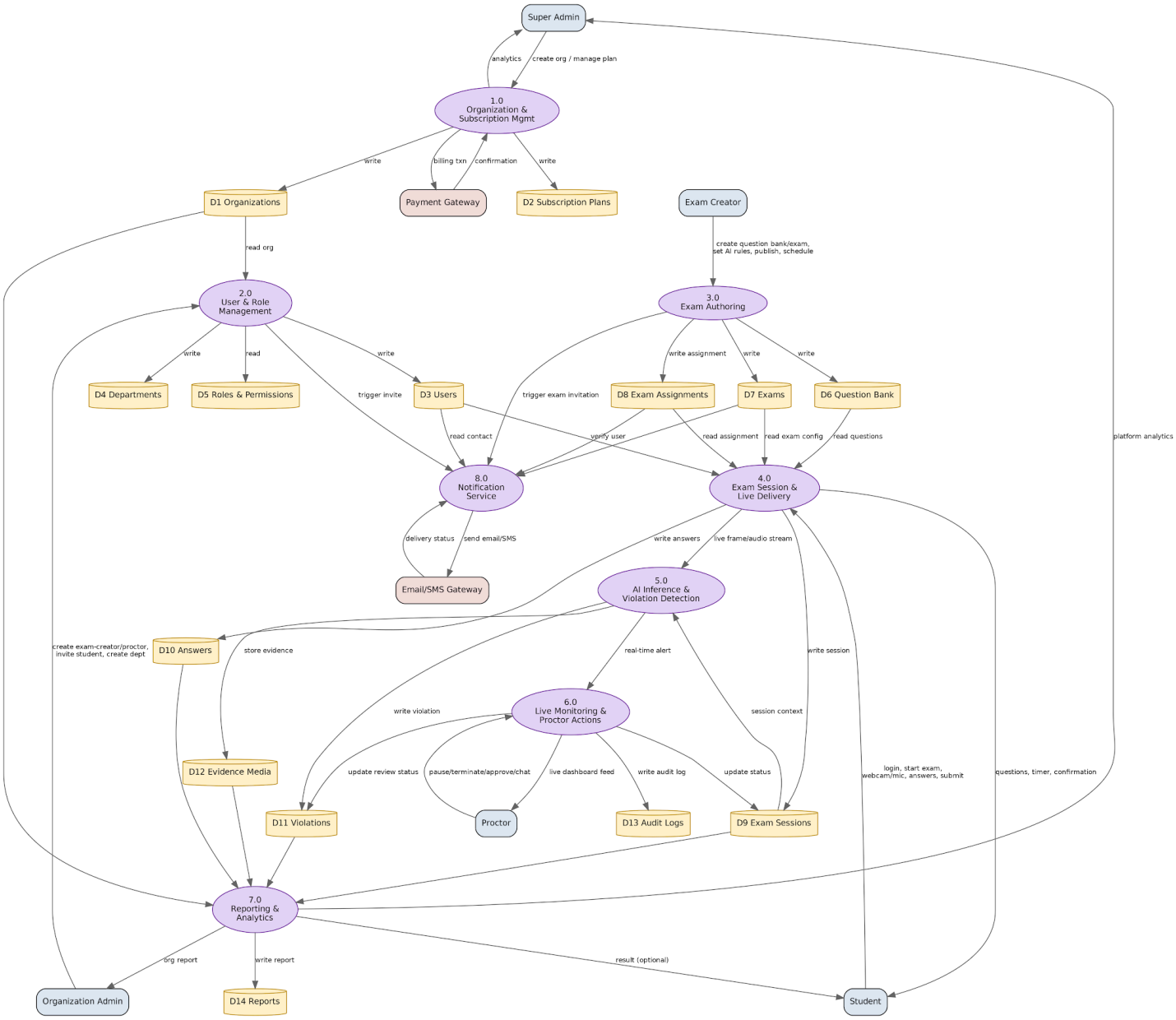


Figure 4 — Level 1 DFD (Major Subsystems)

#	Process	Module(s)
1.0	Organization & Subscription Mgmt	organization
2.0	User & Role Management	auth, user, role
3.0	Exam Authoring	question-bank, exam

#	Process	Module(s)
4.0	Exam Session & Live Delivery	exam-session (Level 2a below)
5.0	AI Inference & Violation Detection	proctoring / AI service (Level 2b below)
6.0	Live Monitoring & Proctor Actions	dashboard, audit
7.0	Reporting & Analytics	report
8.0	Notification Service	notification

9.4 Level 2a — Process 4.0: Exam Session & Live Delivery

This decomposes the exam-taking flow the frontend and backend jointly own, from login through submission (Process 5.0 appears only as a black box; its internals are in Level 2b).

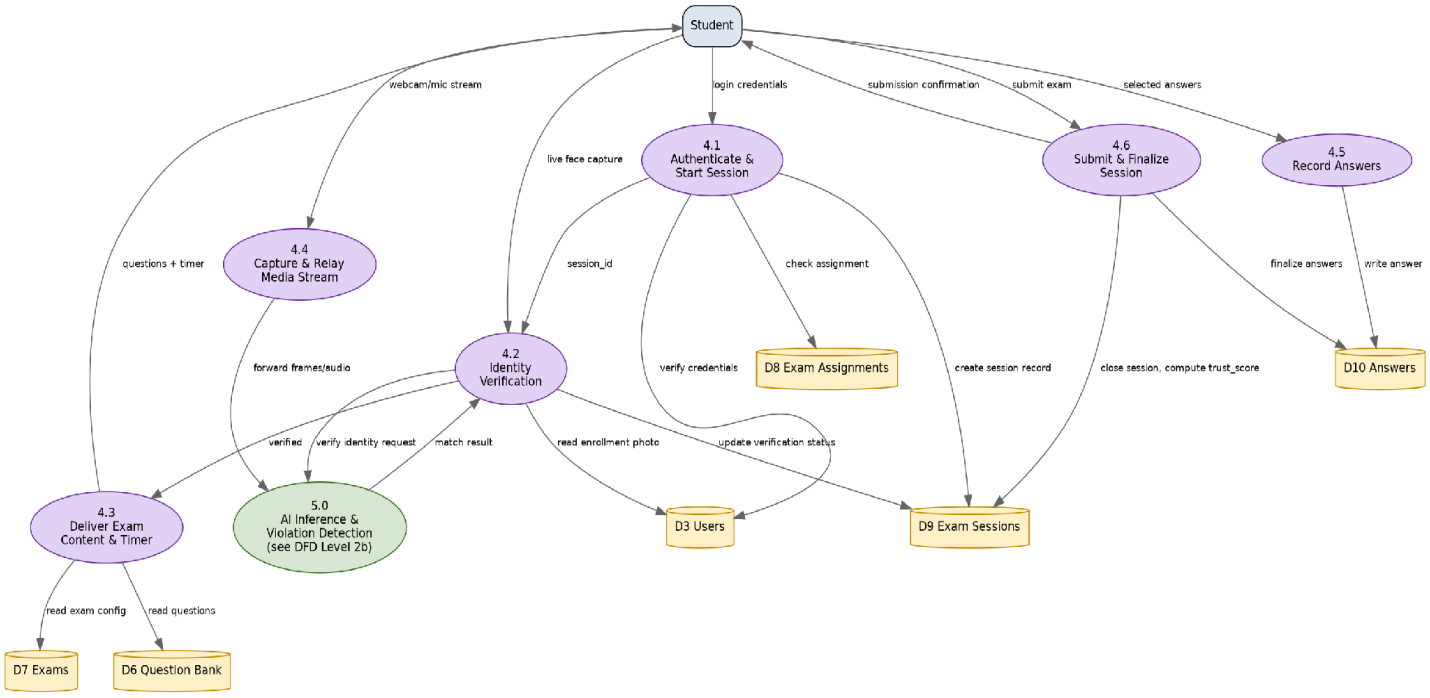


Figure 5 — Level 2a DFD (Exam Session & Live Delivery)

9.5 Level 2b — Process 5.0: AI Inference & Violation Detection

This is the AI/ML subsystem end-to-end — from the moment a frame or audio chunk arrives to the moment a violation is written and the proctor is alerted. Each box maps directly to a function or route in the FastAPI service: 5.1 is the preprocessing pipeline, 5.2–5.5 are the four /infer/* endpoints, and 5.6–5.7 aggregate four model outputs into the one violation record the backend expects.

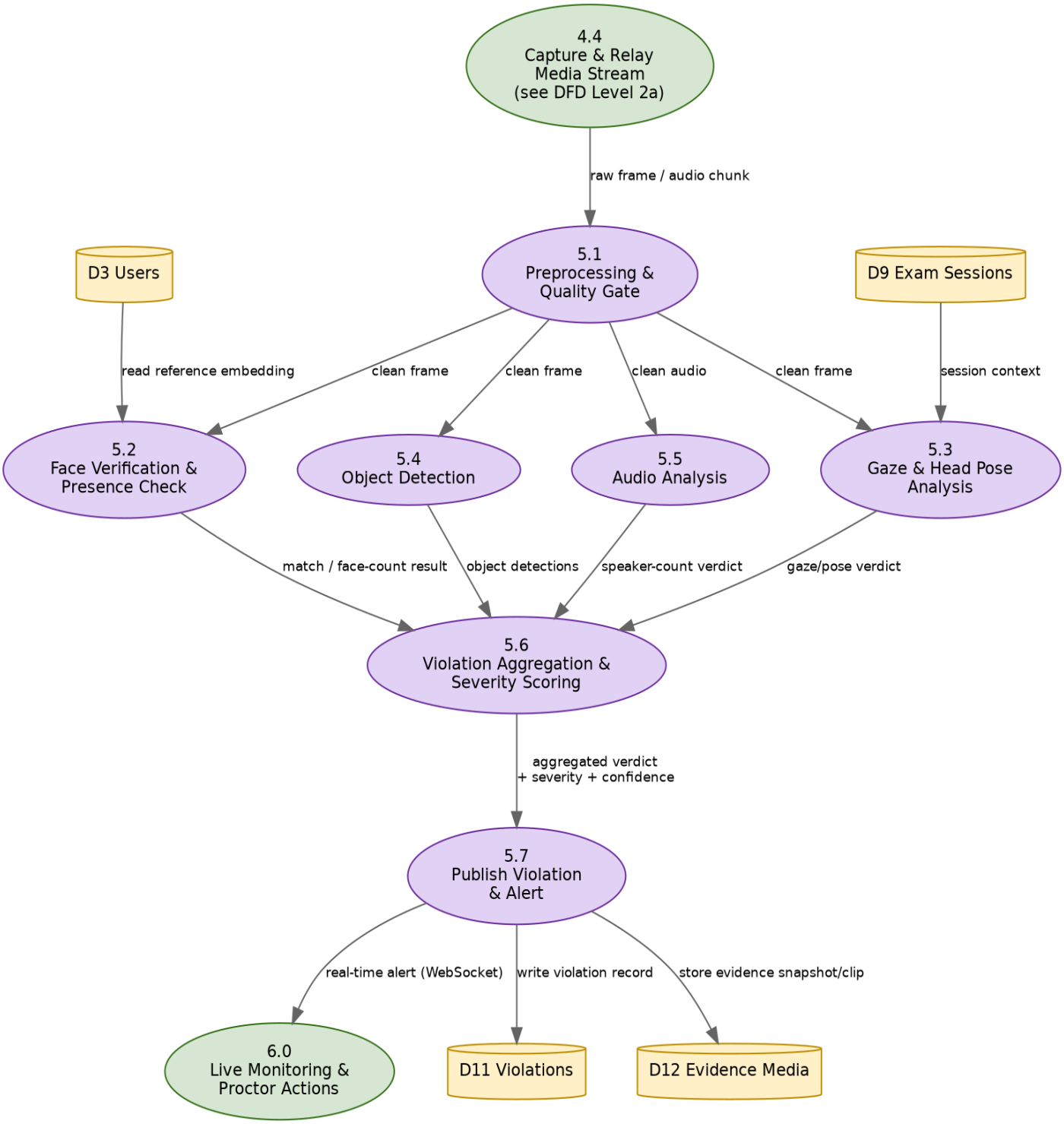


Figure 6 — Level 2b DFD (AI Inference & Violation Detection)

10. Methodology

The proposed system follows a systematic, six-step process for every exam attempt, corresponding directly to sub-processes 4.1–4.6 in the Level 2a data flow diagram (Section 9.4):

Step 1: Authenticate & Start Session: The student logs in with organization-scoped credentials; the backend validates the login and exam assignment, then creates the exam_sessions record.

Step 2: Identity Verification: The system captures a live face image and calls the AI inference service to match it against the student's enrollment photo before the exam is permitted to start.

Step 3: Deliver Exam Content & Timer: Questions are served from the question bank with a live countdown; the frontend continuously relays the webcam/microphone stream to the AI service.

Step 4: Continuous Violation Detection: The AI service analyzes each frame/audio chunk for face presence/match, gaze and head-pose deviation, unauthorized objects, and multiple speakers, aggregating findings into a severity-scored violation.

Step 5: Record Answers & Alert Proctors: Each answer is autosaved as selected; high-severity violations are pushed immediately to the assigned proctor's live dashboard over WebSocket.

Step 6: Submit, Finalize & Report: On submission, the session is closed, the trust score is computed from the full violation history, and a detailed report is generated for review.

Throughout, all user information, examination records, violation logs, and evidence are securely stored with encrypted communication and role-based, organization-scoped access control, so no tenant can access another tenant's data.

11. Features

- **Multi-Tenant Organization Management**: A Super Admin can onboard and manage independent organizations with isolated data and flexible subscription plans.
- **Secure, Role-Based Authentication**: Utilizes JWT-based login for different roles, including Super Admin, Organization Admin, Exam Creator, Proctor, and Student, ensuring secure access control.

- **Face Verification**: Implements live webcam comparisons with registered photographs prior to exams to confirm identity.
- **Real-Time Face & Gaze Monitoring**: Continuously monitors for face presence and detects deviations in gaze or head pose during examinations.
- **Object & Audio Monitoring**: Identifies prohibited objects and multiple audio sources, enhancing exam integrity by monitoring environmental distractions.
- **Browser Lockdown**: Monitors and restricts tab switching, window changes, and attempts to exit fullscreen during exams.
- **Real-Time Alerts & Trust Scoring**: Provides instant proctor notifications for violations, along with trust scores based on recorded infractions.
- **Violation Report Generation**: Creates detailed violation reports including timestamps, evidence, and confidence scores for accountability.
- **Exam Authoring Tools**: Allows Exam Creators to develop question banks and set AI proctoring rules for exam configuration and scheduling.
- **Subscription & Billing**: Features integrated payment gateways for managing organization-level billing through various subscription plans.
- **Notification Service & Audit Logging**: Offers email/SMS notifications and maintains a comprehensive audit trail of administrative actions across tenants.
- **Scalable Microservice Architecture:** Developed using React/Next.js, Spring Boot, FastAPI, PostgreSQL, and Redis, ensuring independent scalability for the system.

12. Hardware & Software Requirements

12.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 (10th Gen+) / AMD Ryzen 5 or equivalent
RAM	8 GB (16 GB recommended for AI model development)
Storage	256 GB SSD or above
Webcam / Microphone	HD webcam (720p+) with built-in or external microphone
Internet Connection	Stable broadband, minimum 10 Mbps
Display	1366 × 768 resolution or higher
Graphics	Integrated graphics for development; dedicated GPU recommended for AI fine-tuning

12.2 Software Requirements & Technology Stack

Each technology below was selected to perform a specific function within the three-tier architecture (Section 7).

Technology	Purpose
React.js / Next.js, TypeScript, Tailwind CSS	Responsive, type-safe exam UI and proctor/admin dashboards
Java (JDK 17+), Spring Boot, Spring Security	Backend microservices, REST/WebSocket APIs, JWT-based auth and RBAC
Python, FastAPI	AI inference microservice and its REST contract with the backend
PostgreSQL	System of record — organizations, users, exams, sessions, violations
Redis	Caching, real-time pub/sub, and frame-stream hand-off to the AI service
WebSocket (Socket.IO)	Real-time alerts between the AI service, backend, and proctor dashboard
OpenCV, MediaPipe	Image processing, face detection, landmarks, gaze and head-pose tracking
ArcFace (InsightFace)	Face recognition / candidate identity verification
YOLOv8	Detection of unauthorized objects (phones, books, devices)
Silero-VAD, pyannote.audio	Voice activity detection and speaker diarization
ONNX Runtime	Optimized, faster AI model inference in production
Docker & Docker Compose	Containerization and consistent multi-service deployment
Git & GitHub	Version control and collaborative development

13. Impact & Expected Outcomes

The AI Exam Proctoring System aims to improve the security, fairness, and reliability of online examinations through real-time AI monitoring, while its multi-tenant SaaS design multiplies that impact across many institutions from a single deployment — smaller institutions that could never build this in-house gain access to enterprise-grade proctoring at a fraction of the cost. By automatically recording violations with timestamps and evidence, the project reduces the workload of human invigilators and gives administrators an objective, trust-score-backed basis for decisions.

13.1 Expected Outcomes

- Secure candidate authentication that accurately verifies identity before the exam begins, reducing impersonation risk.
- Continuous real-time monitoring of webcam and microphone input for the full duration of every exam.
- Accurate, automated detection of multiple faces, face absence, gaze deviation, unauthorized objects, multiple speakers, tab switching, and fullscreen exits.
- Automated, evidence-backed alerts — timestamped and confidence-scored — delivered to proctors in real time.
- Objective trust-score evaluation and comprehensive, exportable examination reports.
- Functional multi-tenant onboarding — a Super Admin can bring a new organization fully online without any code changes.
- Significantly reduced dependence on manual invigilation, and a modular platform ready for future enhancement.

14. Advantages

- **Automated Integrity Checks**: The system conducts continuous checks to detect cheating attempts throughout the examination process.
- **Cost Reduction:** It significantly reduces the need for manual invigilation, cutting costs for institutions.
- **Accurate Identity Verification**: AI-based face recognition ensures high confidence in preventing impersonation.
- **Evidence-based Fairness**: Each violation is timestamped and accompanied by a confidence score and supporting evidence.
- **Multi-tenant SaaS Architecture**: Multiple institutions can share infrastructure, lowering adoption costs.
- **Secure Design**: Features tenant isolation, modern authentication, and encrypted communication to protect data.
- **Location Independence**: Enables secure remote examinations from any stable internet connection.

15. Limitations

- **Hardware & Connectivity Dependenc**: Accurate detection relies on operational webcams, microphones, and stable internet; poor lighting affects results.
- **False Positives/Negatives**: AI may misclassify behaviors or miss genuine violations, but a trust-score model helps mitigate this.
- **Privacy Concerns**: Continuous data capture necessitates compliance with stringent data protection regulations.
- **Compute Costs**: Real-time processing for numerous concurrent sessions requires significant computational resources.
- **Sophisticated Circumvention**: Advanced tactics like deepfake injection pose challenges that may need additional security measures.
- **Onboarding Overhead**: Each new organization requires manual setup, involving branding and proctor assignments.

16. Future Scope

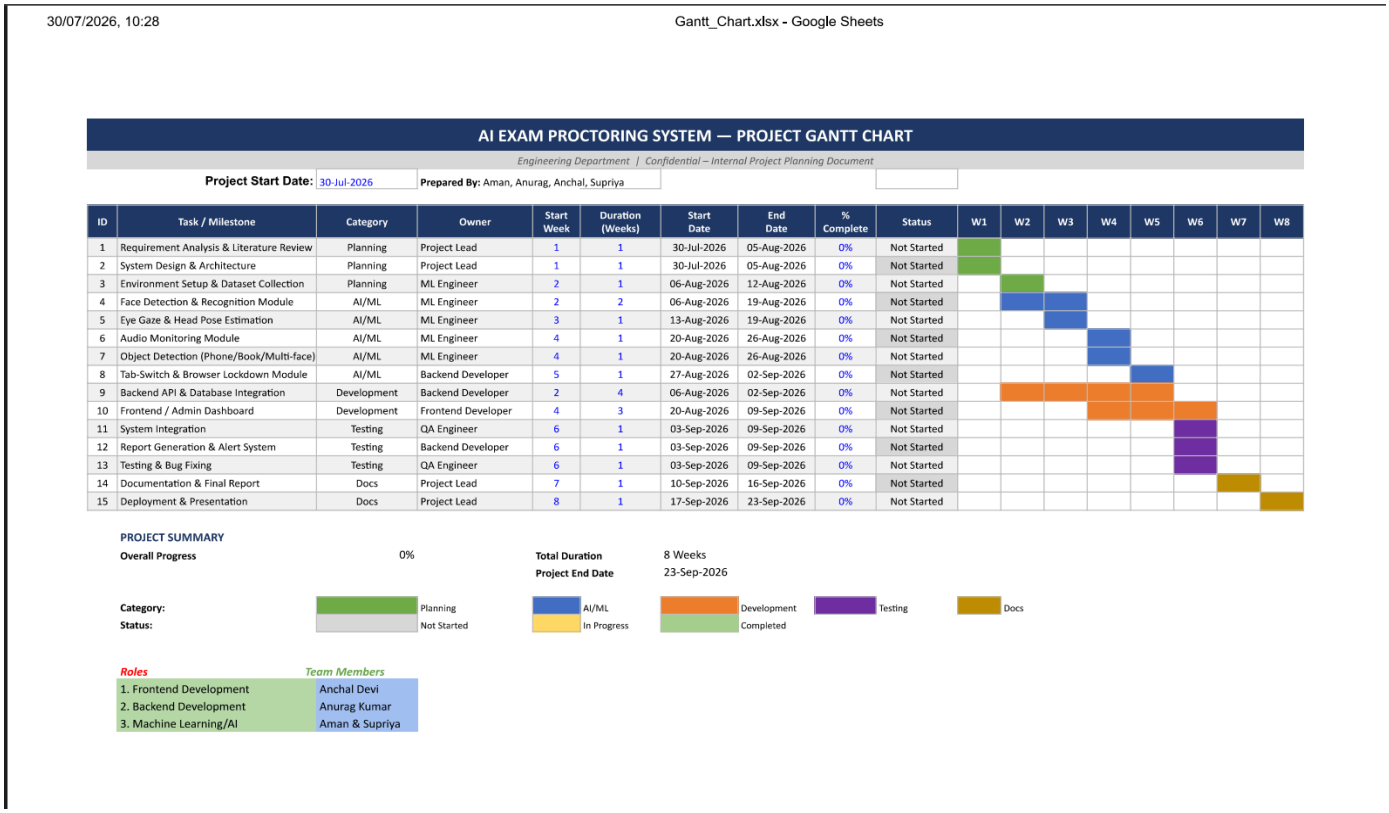
The platform has significant potential for future enhancement without affecting its existing modular architecture:

- **Advanced Liveness Detection**: Implementing 3D face recognition and anti-spoofing technology for enhanced security.
- **Model Accuracy Improvements**: Ongoing refinement of various detection models to minimize errors.
- **Behavioral Analytics:** Incorporating emotion and stress detection for better insights into performance.
- **Integration with LMS**: Enhancing compatibility with platforms like Moodle and Google Classroom for easy examination management.
- **Mobile Support**: Expanding accessibility through mobile and tablet compatibility, along with multilingual options.
- **Scalable Cloud-native Deployment**: Enhancing the AI inference layer to support multiple concurrent users.

- **Continuous Model Retraining:** Utilizing verified examination data for ongoing improvements in detection accuracy.

17. Project Timeline (Gantt Chart)

The project is planned over an 8-week schedule (30 July 2026 – 23 September 2026). AI/ML modules (face detection, gaze/head-pose, audio, object detection) are built in Weeks 2–4 in parallel with backend API and database integration; frontend, tab-switch lockdown, and system integration follow in Weeks 4–6; testing, documentation, and deployment close out Weeks 6–8.



ID	Task / Milestone	Category	Owner	Start Week	Duration (Weeks)	Start Date	End Date	% Complete	Status	W1	W2	W3	W4	W5	W6	W7	W8
1	Requirement Analysis & Literature Review	Planning	Project Lead	1	1	30-Jul-2026	05-Aug-2026	0%	Not Started								
2	System Design & Architecture	Planning	Project Lead	1	1	30-Jul-2026	05-Aug-2026	0%	Not Started								
3	Environment Setup & Dataset Collection	Planning	ML Engineer	2	1	06-Aug-2026	12-Aug-2026	0%	Not Started								
4	Face Detection & Recognition Module	AI/ML	ML Engineer	2	2	06-Aug-2026	19-Aug-2026	0%	Not Started								
5	Eye Gaze & Head Pose Estimation	AI/ML	ML Engineer	3	1	13-Aug-2026	19-Aug-2026	0%	Not Started								
6	Audio Monitoring Module	AI/ML	ML Engineer	4	1	20-Aug-2026	26-Aug-2026	0%	Not Started								
7	Object Detection (Phone/Book/Multi-face)	AI/ML	ML Engineer	4	1	20-Aug-2026	26-Aug-2026	0%	Not Started								
8	Tab-Switch & Browser Lockdown Module	AI/ML	Backend Developer	5	1	27-Aug-2026	02-Sep-2026	0%	Not Started								
9	Backend API & Database Integration	Development	Backend Developer	2	4	06-Aug-2026	02-Sep-2026	0%	Not Started								
10	Frontend / Admin Dashboard	Development	Frontend Developer	4	3	20-Aug-2026	09-Sep-2026	0%	Not Started								
11	System Integration	Testing	QA Engineer	6	1	03-Sep-2026	09-Sep-2026	0%	Not Started								
12	Report Generation & Alert System	Testing	Backend Developer	6	1	03-Sep-2026	09-Sep-2026	0%	Not Started								
13	Testing & Bug Fixing	Testing	QA Engineer	6	1	03-Sep-2026	09-Sep-2026	0%	Not Started								
14	Documentation & Final Report	Docs	Project Lead	7	1	10-Sep-2026	16-Sep-2026	0%	Not Started								
15	Deployment & Presentation	Docs	Project Lead	8	1	17-Sep-2026	23-Sep-2026	0%	Not Started								

Figure 7 — 8-Week Development Gantt Chart

18. Team & Module Ownership

The project team is organized around the three natural technical boundaries of the architecture — AI/ML, Backend, and Frontend — so each member can work independently against the shared API contracts and DFDs in this document, integrating at the checkpoints marked in the Gantt chart above.

Work stream	Owns	Key Technologies	Team Member(s)
AI/ML	Face verification, gaze/head-pose, object detection, audio analysis, violation aggregation (DFD Process 5.0)	Python, FastAPI, MediaPipe, ArcFace, YOLOv8, Silero-VAD, pyannote.audio, ONNX Runtime	Aman, Supriya
Backend	Auth, organization, user, role, question-bank, exam, exam-session, proctoring, violation, report, notification, dashboard, audit, storage (DFD Processes 1.0–4.0, 6.0–8.0)	Java, Spring Boot, Spring Security, PostgreSQL, Redis, WebSocket, Docker	Anurag Kumar
Frontend	Student exam UI, proctor live dashboard, admin/org consoles, Super Admin console	React.js, Next.js, TypeScript, Tailwind CSS, WebRTC client, Socket.IO client	Anchal Devi

19. References

[1] Ultralytics, "YOLOv8 Documentation." Available: https://docs.ultralytics.com/

[2] Google, "MediaPipe Documentation." Available: https://developers.google.com/mediapipe

[3] OpenCV Team, "OpenCV Documentation." Available: https://docs.opencv.org/

[4] InsightFace Developers, "InsightFace: Face Recognition Toolkit." Available: https://github.com/deepinsight/insightface

[5] FastAPI, "FastAPI Official Documentation." Available: https://fastapi.tiangolo.com/

[6] Spring, "Spring Boot Reference Documentation." Available: https://docs.spring.io/spring-boot/

[7] React Team, "React Official Documentation." Available: https://react.dev/

[8] Next.js Team, "Next.js Documentation." Available: https://nextjs.org/docs

[9] PostgreSQL Global Development Group, "PostgreSQL Documentation." Available: https://www.postgresql.org/docs/

[10] Redis Ltd., "Redis Documentation." Available: https://redis.io/docs/

[11] Docker Inc., "Docker Documentation." Available: https://docs.docker.com/

[12] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[13] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2019.